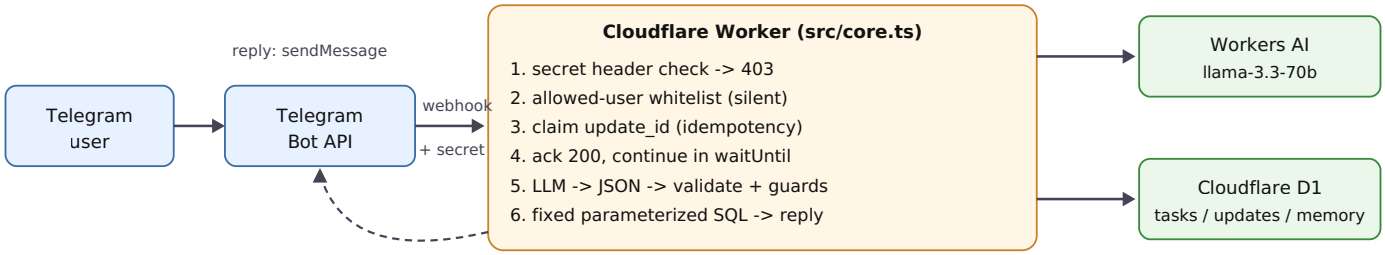
گزارش پروژهٔ hermes-mini — دستیار تلگرامی بدون سرور روی Cloudflare

دانشجو: عماد خسروی • **URL زندهٔ** hermes-mini.www-emadkhosravi999.workers.dev • **تاریخ:** ...

.

۱. معماری نهایی



مسیر یک پیام: Telegram → `POST /webhook` با هدر `X-Telegram-Bot-Api-Secret-Token` → (Worker: ۱) بررسی secret (۴۰۳) ← ۲) بررسی کاربر مجاز (سکوت) ← ۳) ثبت اتمیک `update_id` (جلوگیری از تکرار) ← ۴) پاسخ ۲۰۰ فوری و ادامه در `waitUntil` ← ۵) فراخوانی مدل (Workers AI، `llama-3.3-70b-instruct-fp8-fast`) ← ۶) اعتبارسنجی JSON و گاردها ← ۷) SQL ثابت پارامتری روی D1 ← ۸) `sendMessage`. **کد:** `src/core.ts` (کل منطق، حدود ۷۰۰ خط) و `src/index.ts` (فقط export handler). **داده:** D1 با سه جدول `tasks`، `processed_updates`، `messages` و migration جدا (`migrations/0001_initial.sql`).

۲. توجیه تصمیم‌ها

پرسش	تصمیم و دلیل
Webhook یا Polling؟	**Webhook.** Polling به پروسهٔ همیشه‌روشن نیاز دارد و با Worker و قید «بدون سرور» نمی‌خواند. Webhook فقط هنگام پیام اجرا می‌شود و `secret_token` رسمی تلگرام اصالت فرستنده را می‌دهد. هزینه: URL عمومی HTTPS و پاسخ ۲۰۰ درست برای جلوگیری از redelivery.
چرا FaaS؟	بار کار پراکنده است (چند ده پیام در روز، بیشتر زمان بیکار). FaaS یعنی پرداخت/مصرف فقط هنگام رویداد، بدون مدیریت OS و uptime، با TLS و مقیاس خودکار. معایب پذیرفته‌شده: بدون حافظهٔ پایدار در پروسه، و سقف‌های پلن رایگان (مثل ۱۰ ms CPU).
چرا D1 نه KV؟	داده رابطه‌ای و پرس‌وجودار است (فیلتر کاربر/وضعیت/روز، شناسهٔ یکتا). `INSERT OR IGNORE` اتمیک برای idempotency و `CHECK` و ایندکس در D1 هست؛ KV نهایتاً-سازگار است و عملیات اتمیک چندکلیدی و کوئری ثانویه ندارد.
وضعیت پایدار کجاست؟	فقط D1: `tasks` (داده)، `processed_updates` (idempotency، حذف پس از ۷ روز)، `messages` (حافظهٔ مکالمه، حداکثر ۱۰ پیام و ۵۰۰ نویسه). هیچ حالتی در حافظهٔ Worker نگه داشته نمی‌شود. «امروز» = روز تقویمی `Asia/Tehran` (محاسبه با `Intl`، ذخیره در `created_day/done_day`).

۳. امنیت و لایهٔ LLM

- **امنیت:** secret در Secret Store (نه Git)؛ مقایسهٔ ثابت‌زمان با SHA-256؛ شناسهٔ کاربر مجاز هم Secret است؛ فقط چت private؛ کوئری‌ها همه `.bind()`؛ timeout برای Telegram (۱۰ ثانیه) و مدل (۲۰ ثانیه)؛ لاگ JSON بدون متن پیام و توکن (تست‌شده).
- **قرارداد JSON:** مدل فقط یکی از شش action (`add, done, undone, del, list, chat`) را به‌صورت یک شیء JSON برمی‌گرداند. **کد** آن را با validator دست‌نویس اعتبارسنجی می‌کند (نوع و فیلد الزامی، `task_id` صحیح مثبت، فیلدهای اضافه حذف). مدل هرگز SQL نمی‌نویسد.
- **مسیر امن خطا:** خروجی خراب → یک retry ترمیمی → پیام فارسی بدون هیچ نوشتن در DB؛ timeout یا خطای مدل → پیام فارسی، بدون retry.
- **گاردهای policy (قبل از نوشتن):** ۱) درخواست گروهی («همه رو حذف کن») رد می‌شود حتی اگر مدل اشتباه کند؛ ۲) عدد صریح کاربر باید با `task_id` مدل یکی باشد، وگرنه سؤال تأییدی؛ ۳) `task_id` باید در فهرستی باشد که به مدل داده شده؛ ۴) هر query با `user_id` محدود است.
- **محدودیت صادقانه:** prompt injection کاملاً حذف نمی‌شود؛ فقط اثرش به یک کار معتبر و متعلق به همان کاربر محدود شده. گاردها جلوی نوشتن اشتباه را می‌گیرند ولی کیفیت فهم مدل را بهتر نمی‌کنند.

۴. تحلیل هزینه (جزئیات: COST_ANALYSIS.md)

قیمت و سهمیه‌ها از مستندات رسمی Cloudflare (نوامبر ۲۰۲۶)؛ ردیف‌های D1 روی کد واقعی در شبیه‌ساز **محلی** اندازه‌گیری شد؛ **توکن‌ها، نرخ retry و CPU فرض‌اند** (اندازه‌گیری‌نشده). مدل پیش‌فرض: حدود ۳۶ تا ۸۹ Neuron به‌ازای هر پیام.

	۱ کاربر (۵ / ۲۰ / ۶۰ پیام در روز)	۱۰٬۰۰۰ کاربر (۵ / ۲۰ / ۶۰ پیام)
Neuron در روز (سقف رایگان ۱۰٬۰۰۰)	۱۸۱ / ۱٬۰۷۶ / ۵٬۳۴۲	≈ ۱٫۸ / ۱۰٫۸ / ۵۳٫۴ میلیون
نوشتن D1 در روز (سقف ۱۰۰٬۰۰۰)	۵۵ / ۲۸۰ / ۱٬۰۲۰	۵۵۰ هزار / ۲٫۸ / ۱۰٫۲ میلیون

	۱ کاربر (۵ / ۲۰ / ۶۰ پیام در روز)	۱۰,۰۰۰ کاربر (۵ / ۲۰ / ۶۰ پیام)
درخواست Worker در روز (سقف ۱۰۰,۰۰۰)	۵ / ۲۰ / ۶۰	۵۰ هزار / ۲۰۰ هزار / ۶۰۰ هزار
جا می‌شود در پلن رایگان؟	**بله**	**خیر** (Neuron، نوشتن و خواندن D1، و در typical/max درخواست Worker)
هزینهٔ ماهانهٔ تخمینی	۰$	مدل پیش‌فرض ≈ ۶۰۰$ تا ۱۷,۹۰۰$ (typical ≈ ۳,۶۰۰$)؛ gemma-4-26b ≈ ۱۹۰$ تا ۵,۲۰۰$ (typical ≈ ۱,۱۰۰$)

نتیجه: برای یک کاربر، اولین سقف **Neuron** است (حدود ۱۸۶ پیام در روز با مدل پیش‌فرض)، نه D1. هزینهٔ ۱۰,۰۰۰ کاربر را LLM تعیین می‌کند؛ Workers و D1 ناچیزند. سناریوی ۱۰,۰۰۰ کاربر **فرضی** است: پروژه تک‌کاربره طراحی شده و چنین مقیاسی بازطراحی (چندکاربره، سهمیهٔ هر کاربر) می‌خواهد. ریسک اندازه‌گیری‌نشده: سقف ۱۰ ms CPU روی پلن رایگان.

۵. مشکلات واقعی که رخ داد و راه‌حل‌ها

-
-

#	نشانه	علت	راه‌حل	وضعیت
۱	`npm install` در محیط ساخت: `Cannot read properties of null (reading 'edgesOut')`	vitest ۵ با peer وابستگی پلاگین تست (x.۴) ناسازگار؛ باگ resolver در npm 10.9.7	pin `vitest@4` و `.npmrc` با `legacy-peer-deps`	تأیید: `npm ci` روی کپی تمیز
۲	بستهٔ `vitest-pool-workers` منسوخ‌شده بود	تغییر نام رسمی	`@cloudflare/vitest-plugin`	تأیید
۳	تست‌ها سبز ولی Worker در `wrangler dev` بالا نمی‌آمد	runtime هر export غیر handler (مثل ثابت `DEFAULT_MODEL`) را رد می‌کند	جداسازی `index.ts` و `core.ts` + تست «فقط export پیش‌فرض»	تأیید با `wrangler dev`
۴	Mutation M7 زنده ماند	تست حافظه خروجی خوانده‌شده را می‌سنجید و جدول را نمی‌شمرد	شمارش مستقیم ردیف‌ها	تأیید
۵	`Missing '(' after 'if'` :`set-webhook.ps1`	فایل bash با پسوند ps1	نسخهٔ واقعی PowerShell	ساخته و روی PS 7 تست‌شده؛ ▪
۶	`verify-live.ps1`: همهٔ GETها «got 0»	`[string]$Body=$null` در PowerShell رشتهٔ خالی می‌شود و `-Body` به GET می‌رسید (PS 5.1)	شرط `Length -gt 0` و نمایش علت خطا	**تأیید روی PS 5.1 شما:** ۶ بررسی PASS روی URL زنده
۷	`npm test` فایل‌های تکراری و ۴ فایل «0 test»	Vitest همهٔ `*.test.ts` را می‌یابد: پوشهٔ پشتیبان، کپی تو در تو، کپی‌های ریشه	`include: test/**` و `exclude`	تأیید در شبیه‌سازی؛ ▪
۸	`wrangler tail`: `webhook_forbidden` مکرر	secret ذخیره‌شده در Worker با secret ثبت‌شده در `setWebhook` یکی نبود	secret جدید، `wrangler secret`، تأیید با `verify-live`، ثبت دوبارهٔ webhook	تأیید غیرمستقیم: لاگ بعدی از مرحلهٔ secret ه whitelist گذشت
۹	`db_error … reading 'prepare'`	binding `DB` روی Worker منتشرشده نبود	ساخت D1، `database_id` واق `deploy --dry-run` برای دیدن `env.DB`، migration ریموت، deploy	▪
۱۰	محیط ساخت به Cloudflare و Telegram دسترسی نداشت	محدودیت شبکه و نبود login	استقرار و ثبت webhook توسط خود دانشجو روی دستگاه خودش؛ تست با Mock + smoke محلی	تست‌های Mock تست سرویس واقعی نیستند

۶. وضعیت نهایی و محدودیت‌ها

- **تأییدشده با اجرا:** ۹۱ تست Vitest (workerd + D1 محلی)؛ ۲۲ بررسی smoke واقعی محلی؛ ۹ جهش (mutation) همگی گرفته شد؛ health check روی URL زنده از دستگاه شما (۶ PASS).
- **تأییدنشده:** پاسخ واقعی Workers AI به پیام‌های چالشی فارسی (کیفیت و تأخیر)؛ ارسال واقعی `sendMessage`؛ مصرف CPU و Neuron واقعی؛ [نتیجهٔ دموی زنده].
- **پیاده‌سازی نشده:** بونوس Cron؛ شمارندهٔ سقف روزانهٔ Neuron؛ ثبت `usage` توکن‌ها.